Gemini Clone - Setup Complete ☑

What Was Fixed

1. Server Configuration

☑ Created `server/package.json` with proper dependencies
☑ Fixed `.env` file: Changed `GOOGLE_API_KEY` → `GEMINI_API_KEY` and port to 5000
☑ Fixed `server.js`: Updated dotenv path and ensured multi-turn chat support
☑ Installed all server dependencies

2. Frontend Integration

☑ Created `src/services/geminiApi.js` - API wrapper for backend communication
☑ Created `src/hooks/useGeminiChat.js` - Custom hook for chat state management
☑ Updated `ChatInterface.jsx` - Full integration with backend:

- Form submit handler
- Message display (user and model)
- Loading states
- Error handling
- Enter key to send
- Disabled state during loading

3. Styling

☑ Added custom scrollbar styles to `index.css`
☑ Removed scrollbar arrow buttons

4. Scripts

☑ Updated root `package.json` with proper server script

How to Run

Terminal 1 - Start Backend Server

```
cd server
npm run dev
```

You should see: ☑ `Server running on http://localhost:5000`

Terminal 2 - Start Frontend (React)

```
npm run dev
```

Open browser to the Vite dev server URL (usually http://localhost:5173)

How It Works

1. **User types a message** in ChatInterface
2. **Frontend** adds user message to state, calls `send(text)` from hook
3. **Hook** POSTs to `http://localhost:5000/api/chat` with all messages
4. **Backend** builds chat history, sends last user message to Gemini
5. **Gemini** returns reply
6. **Backend** sends reply back to frontend
7. **Frontend** adds model message to state and displays it

Security Note

⚠ **IMPORTANT**: Your API key `AIzaSyAo9Pa5gLCtEwVZGKifyLOyqhorfuSihrY` is now visible in this conversation.
Regenerate it immediately at https://aistudio.google.com/apikey

Then update `server/.env` with the new key.

Features Now Working

- ☑ Multi-turn chat with context
- ☑ Auto-resizing textarea (up to 5 lines, then scrollable)
- ☑ Custom scrollbar (thin, no arrows)
- ☑ Loading states
- ☑ Error handling
- ☑ Enter to send, Shift+Enter for new line
- ☑ Message display (user vs model styling)
- ☑ Intent cards (clicking fills textarea)

Next Steps (Optional Enhancements)

1. **Streaming** - Add real-time token streaming
2. **Markdown** - Render code blocks and formatting in replies
3. **Copy button** - Allow copying model responses
4. **Clear chat** - Button to reset conversation
5. **Persistent storage** - Save chats to localStorage or database
6. **System prompt** - Add custom instructions for the AI
7. **Rate limiting** - Protect backend from abuse

Troubleshooting

Backend won't start

- Check if port 5000 is available
- Ensure `.env` file exists in `server/` folder

- Verify API key is correct

Frontend can't connect

- Ensure backend is running first
- Check console for CORS errors
- Verify fetch URL is `http://localhost:5000/api/chat`

No response from Gemini

- Check server terminal for error logs
- Verify API key is valid
- Try a different prompt (some may be blocked by safety filters)

File Structure

```
project/
├── server/
│   ├── .env              # Environment variables (API key, port)
│   ├── package.json      # Server dependencies
│   └── server.js         # Express backend
├── src/
│   ├── hooks/
│   │   └── useGeminiChat.js    # Chat state management hook
│   ├── services/
│   │   └── geminiApi.js        # API wrapper
│   └── components/
│       └── feature/
│           └── chat_Interface/
│               └── ChatInterface.jsx  # Main chat UI
└── package.json          # Root dependencies
```

All systems ready! 🚀